

Using wireless cameras to improve computation speed for vision-based line following robots

Matthew King

Abstract—This paper proposes a method for computer-vision line-following. The approach uses the probabilistic Hough line transform to find lines in the frame, then adjusts motor speeds based on relative line angle. The algorithm was tested on a laptop which processed video from a wireless camera and transmitted radio commands to a remote-controlled (RC) tank. The RC tank achieved a maximum speed of 60mm/s with a minimum turning radius of 70mm which is not sufficient for industrial or racing applications. However, the algorithm did operate at a higher frame rate than previous papers by running the computer vision algorithm on a remote device. The method could be improved with additional pre-processing algorithms to remove glare artifacts from the captured images and remove the distortions introduced by the camera's single point perspective. Switching to a centroid-based path finding algorithm is recommended when compared to the relative line angle approach because it generates smoother trajectories and improves handling in corners.

Index Terms—Computer Vision, Line Following Robot, OpenCV, Python

I. INTRODUCTION

Autonomous guided robots (AGVs) are increasingly used across multiple sectors for automating menial, dangerous, and remote work. These robots use a variety of systems for positioning, such as gps, computer vision, markers and radio control [1]. Multiple different sensor types are often combined to overcome their individual limitations using sensor fusion and Kalman filters. Including line following functionality is a common method for using AGVs for indoor applications such as warehouse logistics [2]. Other locating technologies such as relative signal strength or GPS can be unreliable in indoor environments making line following one of the preferred methods for implementing autonomous indoor robots.

Line following robots (LFRS) use sensors to follow lines on the ground. They traditionally use infra-red (IR) sensors which differentiate between a white floor and black line to output a digital high/low signal. The IR sensors are placed strategically so the robot can detect corners, intersections and branches in the line [3]. The LFR adjusts its motor speed by calculating its position relative to the line. This process is continuous, forming a feedback loop which allows the robot to correct for small external disturbances. Racing LFRs has also been used as a form of competition and education, with multiple global competitions being held annually.

LFRs have a limited ability to recover when they lose a line because the IR sensors are traditionally placed close to the ground with a narrow field of view (FOV). Recovery systems are generally simple methods such as driving in a circle or reversing, with commercial systems generally requiring human intervention to recover when the line is lost. LFRs are also

susceptible to any changes in the environment that affect the amount of IR light received by the sensor, such as changes to lighting conditions or dirt on the floor. These challenges only increase with the addition of branched paths and other devices operating in the environment [4]. Computer vision has been suggested as an alternative to traditional sensors because of its ability to extrapolate multiple contextual cues from a single 'sensor' input.

II. BACKGROUND

Cesar et Al designed a convolutional neural network to control a LFR with analog IR sensors as inputs [5]. Their result was able to outperform a Position-Integral-Derivative (PID) based controller in racing which suggests AI is a viable alternative to classical controllers in LFRs. Cesar et Al suggest their AI outperforms the PID controller because it can operate better with non-precise information and different environments.

Shuang-Soh et Al used classical computer vision techniques to design line-following indoor drones [6]. These drones followed yellow lines by calculating the path centroid and used SIFT feature tracking to estimate the drone's coordinates in 3D space. SIFT feature tracking uses corners and visually unique features in sequential images to track changes to the camera's pose over time. Finally a Kalman filter was used to combine the SIFT tracking and a wireless signal strength based position estimate to provide a smoother estimate of the drone's position. Performance was limited by the inadequate processor on the drone, which could only analyse 2.8 frames per second (FPS). With modern 5G video transfer methods providing frame latency as low as 30ms [7] it will be possible for autonomous vehicles to send transmit video data wireless and receive control suitable for real-time control.

A simple implementation of a vision based LFR was analysed by Ma'arif et Al in [8]. This report tested their algorithm in the commercial robotics simulator WeBots. They simulated a Khepera IV robot, using its single front mounted camera for line following. Their robot used contour extraction to calculate the lines centroid and IF-ELSE logic statements for cornering based on the centroid's coordinates relative to the frame's center. Their results were inhibited by the low FOV of the Khepera IV's camera, which prevented the robot from detecting the line far ahead of its current position. The Khepera IV also has several IR sensors specifically for line detection [9], which may have improved their performance.

Combining computer vision and IR sensors into a LFR yields impressive results. Eleftheriou et Al [3] describe a line

LFR capable of speeds up to 2.25 meters per second. This speed was achieved using an array of IR sensors detecting the immediate line and a camera to observe the upcoming line. The LFR uses the camera to observe the upcoming path, and a fuzzy-logic based system is used to adjust the PID controller parameters for different cornering scenarios. They proved including this predicative approach was superior to traditional LFRs by achieving second place in the 12th International Robotic Arena challenge in Poland. Their approach is also supported by other researchers such as Juang et Al [10] who use computer vision for path-finding humanoid robots.

III. METHOD

A. Setup

The algorithms in this report were developed with OpenCV and Python 3.9.0. Tests were conducted on a Macbook Pro 2018 with a 2.2GHz 6-Core Intel i7 running MacOS Sonoma 14.4.1. A discrete GPU was not used because the Macbook does not include one compatible with OpenCV. Video was captured using an iPhone Xr and transmitted to the Macbook via Bluetooth. This video transfer feature is native to macOS Sonoma. The MacBook analyses the video feed to determine the LFRs heading relative to the target line and generates directive speed commands for steering. The speed values are transmitted to a remote-controlled (RC) tank via radio. The tank has two DC drive motors with a maximum straight-line speed of 240 millimetres per second. The tank-drive configuration allows the RC car to pivot on the spot, it has no minimum turning radius.

B. Algorithms

Once a frame is received from the camera, it is filtered with a Gaussian blur to remove noise. The Gaussian blur filter uses a weighted average of nearby pixels which uses weights following a Gaussian distribution. This is good for removing noise introduced by low light levels, which has a Gaussian distribution except in extremely low light levels [11]. The blurring introduced by a Gaussian blur filter is undesirable because it affects the localization of edges for the subsequent Canny edge detection. The degree of blurring depends on the kernel size, which represents the dimensions of the square area from which pixel samples are taken. Figure 1 shows a line after being blurred with a kernel size of 7, which was used in this method.



Fig. 1. Raw Image

Smoothed Image

Canny edge detection finds edges using the gradient of each pixel, therefore preserving each colour channel is unimportant. Each frame is converted to **grey-scale**, which condenses the 3 colour channels into a single intensity channel with values from 0 to 255. This reduces the data size to 1/3 of its original size without compromising accuracy. Subsequently, the grey-scale frame is transformed from a grey-scale to a binary black-and-white image using Otsu's binarisation filter. **Otsu's method** is a dynamic binarisation technique that uses a histogram to set a threshold for black/white. Assuming the histogram is bi-modally distributed, Otsu's thresholding will choose a threshold that maximises the difference between black and white pixels, whilst minimising the variance within the black and white distributions [12]. This algorithm is widely used in other computer vision applications because it can operate without prior knowledge of image composition and is robust against variable lighting conditions. Edge detection is possible without binarising the image, and is necessary to preserve fine detail however, since the LFR only needs to distinguish between a black line on a white floor binarisation can be implemented to improve algorithm speed without compromising accuracy. An example frame is shown in figure 2.

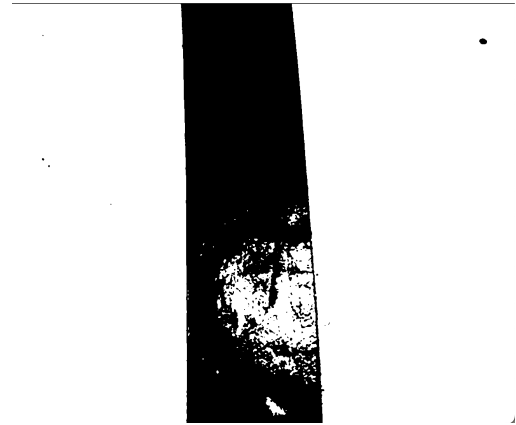


Fig. 2. Binarised Image

To find lines in an image, the first step is locating pixels in the frame likely to be along an edge. These are known as edge points and are detected using the previously mentioned **Canny Edge Detection** algorithm. Canny edge detection begins with another Gaussian blur by default. It can be removed but the efficiency gains are only marginal. Next, each pixel is assigned a gradient magnitude and direction using a combination of Sobel, Prewitt, and Roberts operators. This method uses only the Sobel operator which assigns gradients in the vertical and horizontal directions. This doesn't limit the method's ability to detect corners since it is assumed that all corners will be rounded. A pixel is considered a local maximum if it has the highest gradient magnitude in its gradient direction. By suppressing any edge points not considered a local maximum the edge points are thinned which improves localisation (how closely edge points align with the true edge). Finally, hys-

teresis is used to link edge points together into continuous contours. Any pixel above a high threshold is automatically considered an edge point, those between the high and a low threshold are only considered edges if they are connected to a pixel above the high threshold. Pixels below the low threshold are discarded as edge points. A high threshold of 250 and a low threshold of 50 were used in the proposed method. Figure 3 shows the detected edge pixels from figure 2, note that the reflection from the overhead light has been represented as a cluster of edge points in the line.

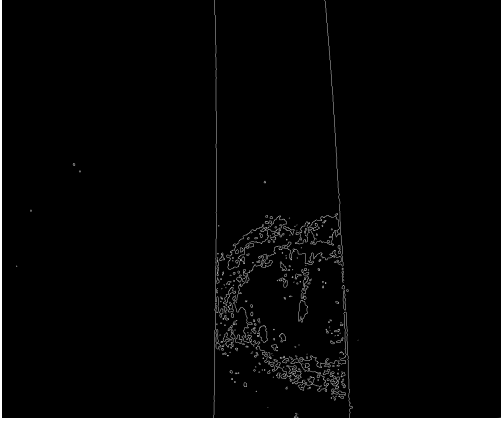


Fig. 3. Edges detected

The **Hough line transform** is used to detect straight lines in an image, even when they are partially obscured or not fully connected. Using the previously obtained edge points, the Hough line transform calculates every possible line that could pass through that pixel, represented by an angle θ and a distance from the frame's origin ρ . This creates a set of sinusoidal curves in "Hough Space" where the coordinate system is (θ, ρ) . Lines are detected by finding sets of curves that intersect in Hough Space, and the parameters of the detected lines are given by the peaks of the Hough Space curves. Once the lines are detected, a threshold is applied to remove weak lines (those with a small number of points), and the coordinates of the remaining peaks are transformed back into Cartesian space to extract the start and stop coordinates of each line segment [13]. Using the Hough line transform is effective however, the large number of edge points made the process computationally expensive and limited the frame-rate to approximately five FPS. The proposed method uses the probabilistic Hough line transform instead which improves performance by only sampling a random subset of the detected edge points. This introduces some instability, as there is theoretically no guarantee that enough of the target line's edge points will be included in the random sample. The proposed method uses a random 25% of the detected edge points, which was determined through testing to be the minimum required for reliable operation. Figure 4 shows the lines detected from the edge points shown in figure 3 and that the edge points erroneously detected from the light reflection have been filtered out by the Hough thresholding. The green

lines are those detected by the transform, and the red one is the line that the control algorithm has selected to follow.



Fig. 4. Lines detected

The robot chooses what lines to follow based on three criteria; Length, start position, and relative line angle. Relative line angle is calculated by equation 1, with a -180 degree phase shift if the result is greater than 90 degrees. Any lines below the minimum length threshold are removed by the prior Hough transform, so the line with at least one coordinate point in the lower half of the image with the minimum relative angle is selected as the target to follow. Choosing lines that 'start' in the bottom half of the image was done to prevent the RC tank from following lines detected at the edges of the camera's FOV. By following the line with the smallest relative angle, the tank continues to travel in a straight line until cornering is the only possible option which prevents algorithm from following branching parts. The RC tank uses DC motors, which are controlled using an H-Bridge. H-Bridges are simple electrical components that provide speed and direction control for DC motors by regulating the applied voltage and current direction. Motor speed is controlled by sending a digital pulse-width-modulated (PWM) signal to the H-Bridge from the RC car's microcontroller. By switching the digital PWM signal at a fixed frequency, the ratio of time spent at logic high compared to logic low encodes the desired value. This gives PWM values a range from $[0, 100]$ %, where 0% represents a stationary motor and 100% is the maximum forward speed. A second digital signal sent by the microcontroller to the H-Bridge is used to specify the motor direction so the computer vision algorithm transmits speed values in a range from $[-100, 100]$. These are transmitted over radio as a two 8 bit signed integers in two 3 byte radio payload. Radio transmission occurs every time a new speed value calculated but has a maximum rate of 100Hz. The RC tank sends a reciprocal transmission of containing the currently set speed values for error checking and to ensure synchronisation. Equation 2 shows how the relative line angle is used to calculate the target speed for each motor. The speed control is open-loop as the RC tank has no way to measure the motor speeds. However, the computer vision algorithm is fast enough to correct for any deviation

Corner Radius (mm)	Pass Rate
200	100%
150	100%
100	83%
90	83%
80	50%
70	50%
60	33%
50	0%

TABLE I
CORNERING RESULTS

introduced by inaccurate speed control.

$$\theta = \arccos \left(\frac{1}{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}} \begin{bmatrix} x_2 - x_1 \\ y_2 - y_1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right) \quad (1)$$

$$v_{L/R} = v_{fwd} \pm \frac{(\theta - \theta_{min}) * (v_{max} - v_{min})}{(\theta_{max} - \theta_{min})} + v_{min} \quad (2)$$

IV. RESULTS

The RC tank used to evaluate the method performance is shown in figure 5. The iPhone 10 affixed to the tank's front was used as a wireless Bluetooth camera, which sent a real-time 1080p video feed to the MacBook at 30 FPS. The CPU was able to process an average of 18 FPS, exceeding the 2.8 FPS result from Shaung-Soh et Al [6]. This represents a 6.4x increase in frame analysis speed, making this result more responsive despite the latency introduced by bluetooth video transmission and radio communications. Frame rate was measured using macOS's hardware monitor. The RC tanks' ability to turn was characterised with right-angle corners of different radii made using electrical tape. Each corner was attempted 3 times for both a left and right turn, with a base forward speed of $v_{fwd} = 30mm/s$. From the results shown in table I the minimum turning radius was determined to be 70mm. There was no difference between the failure rates for left and right turns. The primary failure mode was under-steer, shown in figure 6. Over-steer and parallax error were also present when attempting multiple corners sequentially. Maximum speed was determined by navigating a 150mm radius 90-degree corner and increasing the baseline forward speed til the failure occurred. Speed was measured by driving the tank in a straight line alongside a ruler, and timing how long it took to travel a distance of 300mm. When the line was removed from frame, the dynamic binarisation would behave erratically as the image histogram was no longer bimodally distributed. small scratches in the floor would be detected as lines causing the tank to follow random paths. If the camera found the line again it would attempt to follow it, but this would introduce a large amount of parallax error.

V. CONCLUSION

The fastest forward speed recorded during testing was 50 mm/s, a poor result compared the racing line following robots in [3] which exceeded 2.25 meters per second. This paper's

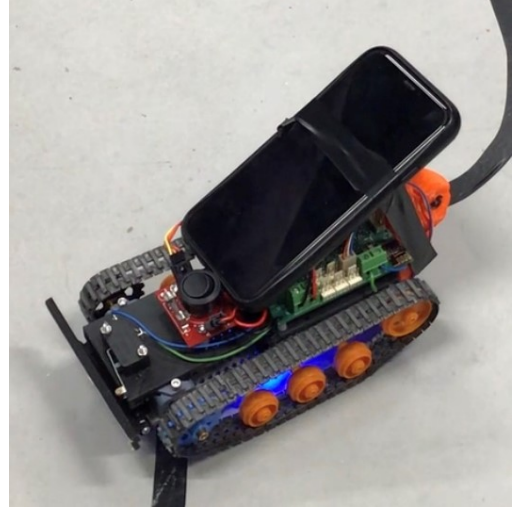


Fig. 5. RC Tank following line

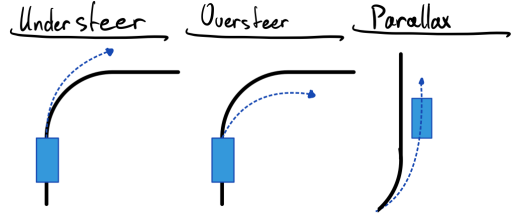


Fig. 6. Failure modes

method was limited by the mechanical constraints of the tank (maximum forward speed of 240mm/s) and the proportional-only control system, which can only provide a linear response unlike the PID system implemented by Cesar et Al. This report's tank couldn't navigate curves with a radius $< 70mm$, again a poor result when compared to the sharp corners executed by [3]. This was due to the poor algorithm choice, which could not correct for the parallax error described in figure 6. Parallax error occurred when the robot travelled parallel to the target line without attempting to recenter itself because the relative line angle is zero. The centroid based approaches discussed in [6] would not have had this issue. This problem is not present for IR line following robots [14] because their narrow field of view ensures the line is always centred on the robot, however this narrow field of view also means parallax error is likely to cause complete failure as soon as the robot loses the line. Using the wireless camera facilitated computer vision on an autonomous vehicle that otherwise would not have had enough processing power to support the algorithms used. This provides a solution to the problems faced by [6], which could also be further improved by utilising the IR sensors and fuzzy logic implemented by [3].

A. Future Research

As shown from the Canny edge points in figure 3 a number of false edge points are introduced from the reflection on the

line's surface. This could be removed using morphological operations such as open or close. Morphological operations erode and dilate contours in a binarised image by removing and adding pixels respectively from contour boundaries [15]. dilating the line's contour to fill the 'hole' left by the glare artifact then using erosion to restore the lines original shape would solve this problem. This would be done after the image had been binarised but before edge detection occurs.

The under and over-steer failure modes can be partially attributed to the camera's single point perspective distorting the line's angle. Because of this distortion, lines parallel to the tank's heading appear to converge which creates an error when the relative angle is calculated. Jianye et Al overcome this problem by using a perspective transform to make the image appear as if it was taken from an overhead camera, instead of one mounted to the robot at an angle [16]. This increases the accuracy of turning signals by removing distortions introduced by the camera's perspective. The perspective transform would improve the relative line angle algorithm, but this algorithm is still not well suited for LFRs.

The proposed method can only detect straight lines, which limits its ability to create smooth turning paths as each corner is approximated by a series of small lines. Furthermore, if the line length threshold is set too high, corners can be missed entirely. This is why it is more common for LFRs using computer-vision to instead extract the line's contour and calculate its centroid. Juang et Al implement this with a recursive box search to calculate multiple centriods along the line to generate smooth curves [17]. After the line contour is detected, the frame is segmented into many smaller boxes each containing a length of the line contour. A smoothed approximation of the original line is obtained using a polynomial regression to fit a curve between each centroid, hence the name "recursive box search". Provided the camera can see enough of the path ahead, this could be combined with the predictive speed control from [3] to provide a smoother trajectory and potentially higher speeds.

REFERENCES

- [1] O. Gumus, M. Topaloglu, and D. Ozcelic, "The use of computer controlled line follower robots in public transport," presented at the 12th International Conference on Application of Fuzzy Systems and Soft Computing, ICAFS, 2016, pp. 202–208. DOI: 10.1016/j.procs.2016.09.390.
- [2] A. Siddiqui, A. Basker, M. Dias, K. Chavan, and N. Avhad, "Line following cart (lfc) for office applications," presented at the 5th Biennial International Conference on Nascent Technologies in Engineering (ICNTE), 2023, pp. 1–6. DOI: 10.1109/ICNTE56631.2023.10146691.
- [3] G. Eleftheriou, L. Doitsidis, Z. Zinonos, and S. A. Chatzichristofis, "A fuzzy rule-based control system for fast line-following robots," presented at the 16th International Conference on Distributed Computing in Sensor Systems (DCOSS), 2020, pp. 388–395. DOI: 10.1109/DCOSS49796.2020.00068.
- [4] A. Toumpa, A. Kouris, F. Dimeas, and N. Aspragathos, "Control of a line following robot based on fsm estimation," presented at the 12th IFAC Symposium on Robot Control (SYROCO), vol. 51, 2018, pp. 542–547. DOI: 10.1016/j.ifacol.2018.11.573.
- [5] C. Minaya, R. Rosero, M. Zambrano, and P. Catota, "Application of multilayer neural networks for controlling a line-following robot in robotic competitions," *Journal of Automation, Mobile Robotics and Intelligent Systems*, pp. 35–42, 2024. DOI: 10.14313/jamris/1-2024/4.
- [6] L. Shuang-Soh and H. Woei-Ho, "Autonomous navigationn of micro air vehicles in warehouses using vision-based line following," 2023. DOI: 10.48550/arXiv.2310.00950.
- [7] S. Song, J. Jung, M. Choi, C. Lee, J. Sun, and J.-M. Chung, "Multipath based adaptive concurrent transfer for real-time video streaming over 5g multi-rat systems," *IEEE Access*, vol. 7, pp. 146 470–146 479, 2019. DOI: 10.1109/ACCESS.2019.2945357.
- [8] A. Ma'arif, A. Nuryono, and Iswanto, "Vision-based line following robot in webots," presented at the 2020 FORTEI-International Conference on Electrical Engineering (FORTEI-ICEE), 2020, pp. 24–28. DOI: 10.1109/FORTEI-ICEE50915.2020.9249943.
- [9] J. M. Soares, I. Navarro, and A. Martinoli, "The khepera iv mobile robot: Performance evaluation, sensory data and software toolbox," presented at the Second Iberian Robotics Conference. Advances in Intelligent Systems and Computing, 2015, pp. 767–776. DOI: 10.1007/978-3-319-27146-0_59.
- [10] L.-H. Juang and J.-S. Zhang, "Visual tracking control of humanoid robot," *IEEE Access*, vol. 7, pp. 29 213–29 222, 2019. DOI: 10.1109/ACCESS.2019.2901009.
- [11] K. Wei, Y. Fu, J. Yang, and H. Huang, "A physics-based noise formation model for extreeme low-light raw denoising," presented at the 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 2755–2764. DOI: 10.1109/CVPR42600.2020.00283.
- [12] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer Computer Science, 2022, pp. 364–368.
- [13] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer Computer Science, 2022, pp. 381–382.
- [14] M. Shah, V. Rawal, and J. Dawlwadi, "Design implementation of high-performance line following robot," presented at the International Conference on Transforming Engineering Education (ICTEE), 2017, pp. 1–5. DOI: 10.1109/ICTEED.2017.8586107.
- [15] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer Computer Science, 2022, pp. 110–111.
- [16] J. He, S. Sun, G. Wang, and C. Zhang, "Lane detection for track-following based on histogram statistics," presented at the 2019 IEEE International Conference

on Electron Devices and Solid-State Circuits (EDSSC), 2019, pp. 1–2. DOI: 10.1109/EDSSC.2019.8754094.

- [17] L.-H. Juang and J.-S. Zhang, “Robust visual line-following navigation system for humanoid robots,” *Artificial Intelligence Review: An International Science and Engineering Journal*, vol. 53, pp. 653–670, 2020. DOI: 10.1007/s10462-018-9672-9.